02-750 Week 7

Automation of Scientific Research

Aidan Jan

March 10, 2026

IWAL Algorithm

"Importance Weighted Active Learning"

- Key Features
  - IWAL is an algorithm for learning **binary and multi-class classifiers**
  - It is a Type I algorithm (i.e., hypothesis elimination)
  - It can be used with any data access model
  - It can be used with **multiple loss functions** (Note: CAL, A^2, and DHM assume 0-1 loss)
  - It handles non-separable data
  - It is mellow and compensates for sampling bias using importance weighting
- Guarantees
  - IWAL will converge to the optimal model
  - IWAL bounds the number of samples needed to reach the optimal model

IWAL Algorithm

```
Set S_0 = {}
for t from 1, 2, ... until data stream runs out
    Receive x_t
    set p_t = rejection-threshold(x_t, history {x_i, y_i, p_i, Q_i : 1 <= i < t})
    flip a coin Q_t in {0, 1} with E[Q_t] = p_t.
        If Q_t = 1, request y_t and set
            S_t = S_{t - 1} + {(x_t, y_t, p_min / p_t)}
        Else
            S_t = S_{t - 1}
    Let h_t = argmin_{h in H} sum_{x, y, c in S_i}(c * l(h(x), y))
```

- The arguments for IWAL include a subroutine that returns the probability of labeling x_t
  - For the purpose of computing importance weights, $p_t = P_{train}(x)$
  - $p_t > 0 \forall x_t$ and will be different for each x_t, in general
  - The paper presents two options for defining this subroutine; however, there are other options
- The p_{min} argument is required for technical reasons.
  - It is a constant $0 < p_{min} < 1$
- The flip coin step will label x_t with a probability p_t randomly. If the algorithm does label x_t, it adds it to set S with importance weight $\frac{p_{min}}{p_t}$

- The last step learns an importance-weighted classifier, where $c = \frac{p_{min}}{p_t}$
  - The importance-weighted estimate of the loss after T rounds is:

$$L_h(h) = \frac{1}{T} \sum_{t=1}^{T} \frac{Q_t}{p_t} l(h(x_t), y_t)$$

  - The authors show that $\mathbb{E}[L_T(h)] = L(h)$, refer to paper for details

Rejection Threshold Subroutines

Approach 1: Loss-weighting

- First, find the model that minimizes the importance-weighted loss

$$L^*_{t-1} = \min_{h \in H_{t-1}} \frac{1}{t-1} \sum_{i=1}^{t-1} \frac{Q_i}{p_i} l(h(x_i), y_i)$$

- Next, keep any model that isn't significantly worse than the best model

$$H_t = \{h \in H_{t-1} \ : \ \frac{1}{t-1} \sum_{i=1}^{t-1} \frac{Q_i}{p_i} l(h(x_i), y_i) \leq L^*_{t-1} + \Delta_{t-1}\}$$

  - Δ_{t-1} is a generalization bound, like A^2 and DHM.
- Finally, return $p_t = \max_{f,g \in H_t, y \in Y} l(f(x), y) - l(g(x), y)$

The rejection probability for x is the largest difference in losses between any pair of models in the version space, considering each potential label for x.

A large difference reflects a significant disagreement between the models, and thus an opportunity to reduce the size of the version space

Approach 2: Bootstrapping

This subroutine is easier to implement than the previous one.

- Parameters: b (length of initial boostrap sample), k (number of predictors used in voting), p_{min} (lower bound on rejection threshold)
- First, if $t \leq b$, set $p_t = 1$
- If $t = b$, train predictors $h_1, \ldots, h_k$ on the initial sample $(x_1, y_1, c_t), \ldots, (x_b, y_b, c_t)$. Denote the set by H. (This is constructing a committee.)
- If $t > b$, set $p_t = p_{min} + (1 - p_{min})[\max_{h_i, h_j \in H, y \in Y} L(h_i(x), y) - L(h_j(x), y)]$
- Return p_t

Approach 3: Critical Importance Weight

This subroutine is even easier to implement than the previous one.

- Let $h = Learn(S)$ be the importance-weighted model trained on S.
- Let $h' = Learn(S \cup \{x, \neg h(x), w_\Delta\})$ be a model trained with x assigned to some label other than the one predicted by h, and using importance weight w_Δ

- Search for w^*_Δ, which is the smallest w_Δ such that $h(x) \neq h'(x)$
- Set $p_t = \frac{1}{w^*_\Delta}$

You can think of w^*_Δ as being the amount of 'energy' needed to create a model h' that would assign x to a different label than model h. If w^*_Δ is small, then it must be easy to find a model that disagrees with h. If w^*_Δ is large, then it must be hard to find a model that disagrees with h.

IWAL Guarantees

- Convergence (Assuming that $p_t > 0, \forall x \in \mathcal{X}$)
  - IWAL will return a near-optimal model, with probability at least $1 - \delta$
  - Notice that we avoid the consistency issue from earlier, even though we have a biased set of labels because $p_t > 0$
- Sample Complexity
  - Like A^2 and DHM, the label complexity depends on the amount of noise
  - The key result is that IWAL is at worst a constant factor than standard supervised learning (i.e., passive learning) in terms of sample complexity
  - The empirical results suggest that IWAL is often much better.

Summary

A^2 and DHM are essentially versions of CAL for non-separable data

- A^2 significance is mainly that it was the first active learning algorithm to be able to handle non-separable data and still provide guarantees with respect to convergence and label complexity
  - Unfortunately, it can't be implemented exactly (for most hypothesis spaces)
  - Running it using sampling is computationally expensive
- DHM also handles non-separable data and comes with guarantees
  - Practical and relatively easy to implement

IWAL also handles non-separable data and comes with guarantees

- Like A^2 and DHM, the label complexity depends on the amount of noise
- IWAL is at worst a constant factor than standard supervised learning (i.e., passive learning) is terms of sample complexity
- The empirical results suggest that IWAL is often much better

The ZLG Algorithm

Key features:

- ZLG is an algorithm for learning **binary classifiers**
- It is a Type II algorithm (i.e., cluster exploitation)
- It can be used with **pool-based selective sampling**
- It is **aggressive**
  - It approximately minimizes expected risk
- Guarantees: None

Assumptions

- **Key assumption:** if two unlabeled instances, x_i and x_j, are similar, then they should have the same label
- There are many ways we might measure similarity. The ZLG paper uses a **radial basis function**.
  - Let $x \in \mathbb{R}^m$. The similarity is measured as follows:

$$w_{ij} = \exp\left(-\frac{1}{\sigma^2}\sum_{d=1}^{m}(x_i(d) - x_j(d))^2\right)$$

    w_{ij} will be large if x_i and x_j are similar, otherwise, w_{ij} will be small

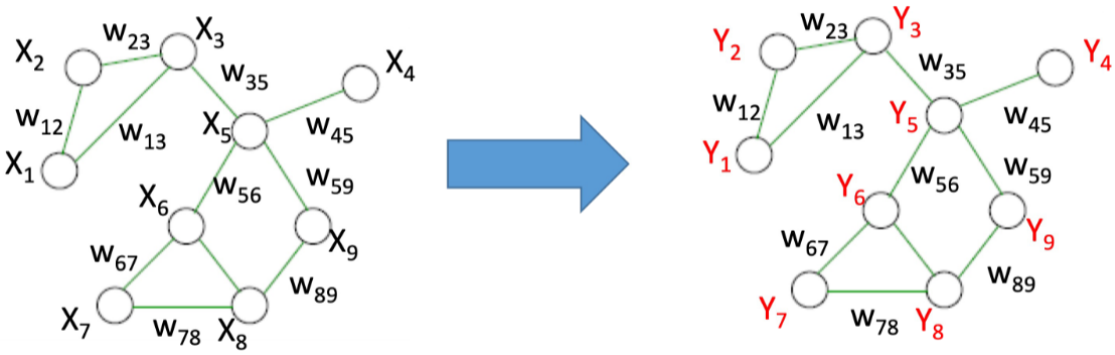
Algorithm

ZLG builds a graph over all instances in the pool using weights

- Let $G = (V, E, W)$ be a weighted graph where:
  - There is one node for each of the n instances in the pool
  - Nodes i and j are connected by and edge if $w_{ij} \geq t$
  - Here, t is a user-defined threshold
  - W is a $n \times n$ weight matrix such that

$$W(i,j) = \begin{cases} w_{ij} & \text{if } w_{ij} \geq t \\ 0 & \text{otherwise} \end{cases}$$

ZLG then considers the labels of the nodes in the graph.



- Labels are binary (0 or 1)
  - This implies there are 2^n possible joint labelings on a graph with n nodes
  - But, under our assumption that if unlabeled instances x_i and x_j are similar, they should have the same label, **some labelings are more likely than others**

We can formlaize our assumption by defining a model for the joint probability distribution over the labels, $P(Y_1, \ldots, Y_n)$

- 'Good' labelings will have high probability under $P(Y_1, \ldots, Y_n)$
- The most obvious choice for specifying $P(Y_1, \ldots, Y_n)$ is as a multivariate **binomial** (i.e., a set of coupled binomial variables), but the authors use a **multivariate Gaussian distribution** instead
  - This is done for mathematical/computational convenience and tractablility
  - This simplification implies that each Y_i is now continuous and that its marginal distribution is Gaussian
  - If we are asked to make the prediction for some particular Y_i, we first infer its value (under the multivariate Gaussian) and then round to 0 or 1

Aside: Multivariate Gaussian Distributions

- Let $Y = \{Y_1, \cdots, Y_n\}$ be a set of n Gaussian random variables
- Let $\mu = \{\mu_1, \cdots, \mu_n\}$ be a mean vector over Y
- Let Σ be the covariance matrix over Y
- Let Σ^{-1} be the precision matrix over Y
- The probability density of a vector $y \in \mathbb{R}^n \sim \mathcal{N}(\mu, \Sigma)$ is:

$$P(y) = \frac{1}{\sqrt{(2\pi)^n |\Sigma|}} \exp\left(-\frac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)\right)$$

  where $|\Sigma|$ denotes the **determinant** of Σ

Overview of the ZLG Algorithm

- Let $Y_u \subseteq Y = \{Y_1, \cdots, Y_n\}$ be the set of **unknown labels**
- Let $Y_k \subseteq Y = \{Y_1, \cdots, Y_n\}$ be the set of **known labels**. Naturally, $Y_u \cap Y_k = \emptyset$
- The ZLG algorithm uses the graph and any known labels to construct the probability distribution, $P(Y_u|Y_k)$
- It then solves for μ_u
  - The elements of μ_u can be rounded to 0 or 1 to make predictions
  - Alternatively, use the elements of μ_u to select the next instance to query

ZLG defines $P(Y_u|Y_k)$ as a **Boltzmann distribution**

- That is, the probability of any vector y is inversely proportional to the user-defined **energy function**
- The energy used by the ZLG algorithm is:

$$E(y) = \frac{1}{2} \sum_{i,j} w_{ij} (y_i - y_j)^2$$

- This is a **harmonic potential**. It penalizes in proportion to the product of the edge weight (if any) and the square of the differences in labels
- This energy can be rewritten in matrix form:
  - Recall that W is the $n \times n$ weight matrix
  - Let D be a diagonal matrix such that $d_i = \Sigma_j w_{ij} = \Sigma_j W(i,j)$. Thus, d_i is the weighted degree of node Y_i
  - Let $\Delta = D - W$ be the Laplacian Matrix
  - The energy in matrix form is $E(y) = y^T \Delta y$

It can be shown that the following is a multivariate Gaussian

$$P(y) = \frac{1}{Z_\beta} \exp(-\beta E(y))$$

where β is the 'inverse temperature' parameter, and

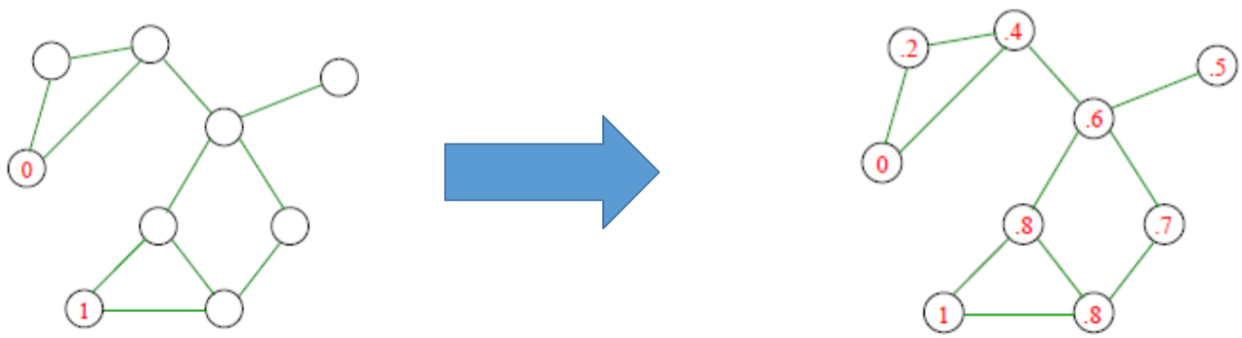
$$Z_\beta = \int \exp(-\beta E(y))\, \mathrm{d}y$$

is the partition function (i.e., normalization constant)

- Notice that:
  - The lowest energy will have the highest probability (Boltzmann dist.)
  - In the absence of *any* known labels, there are two minimum-energy solutions, one where everything is labeled 0, and one where everything is labeled 1.
  - The formal name for their model is a **Gaussian Random Field**, which is a **probabilistic graphical model** (PGM) for encoding multivariate Gaussian distributions

Suppose that we do have some labels

- Conceptually, we could imagine a 'wave' of 0's (resp. 1's) diffusing throughout the weighted graph emanating from the instances labeled 0 (resp. 1's)
- When these waves meet, they will combine to create some intermediate label between 0 and 1, depending on how far away they are from known labels.



Implementation of ZLG

It can be shown that this diffusion process can be implemented efficiently in a Gaussian Random Field in order to infer labels.

- Reorganize the Laplacing Matrix as follows:

$$\Delta = \begin{bmatrix} \Delta_{ll} & \Delta_{lu} \\ \Delta_{ul} & \Delta_{uu} \end{bmatrix}$$

  where $\Delta_{ll}, \Delta_{ul}, \Delta_{lu}, \Delta_{uu}$ are the submatrices corresponding to the groups of labeled (l) and unlabeled instances (u).
- The minimum-energy solution for the unlabeled instances can be computed as $f_u = -\Delta_{uu}^{-1}\Delta_{ul}f_l$, where f_l is the vector of known labels
- Also, the estimated distribution over the unlabeled instances is:

$$P(y_u|f_l) = \mathcal{N}(f_u\Delta_{uu}^{-1})$$

- Thus, we can compute the most likely labels using matrix operations!

Query Selection

- Notice that the predicted labels *could* be used to perform uncertainty sampling, but we already know the problems with that approach
- Instead, the authors perform expected risk minimization
  - Let $f = [f_l, f_u]$ be the minimum energy solution

- Let $\hat{R}(f) = \sum_{i=1}^{n} \mathbb{I}[round(f_i) \neq 0](1 - f_i) + \mathbb{I}[round(f_i) \neq 1]f_i = \sum_{i=1}^{n} \min(f_i, 1 - f_i)$ be the **estimated risk**, where $round(f_i)$ is the result of rounding to 0 or 1.

- Recall that we round each f_i to 0 or 1. Thus, if $f_i > 0.5$, we predict node i is 1. f_i is, therefore, our confidence that node i's label is 1. The estimated risk (i.e., generalization error) is the sum of the probabilities that the rounding selects the wrong label.

- If we request the label for node k and we learn that its label, y_k, the updated estimated risk is:

$$\hat{R}(f) = \sum_{i=1}^{n} \min\left(f_i^{+(x_k,y_k)}, 1 - f_i^{+(x_k,y_k)}\right)$$

- Since we do not know which label the oracle will return, we compute the expected estimated risk if we were to request the label for node k

$$\hat{R}(f^{+x_k}) = (1 - f_k)\hat{R}(f^{+(x_k,0)}) + f_k\hat{R}(f^{+(x_k,1)})$$

- The algorithm queries the node satisfying: $k = \arg\min_{k'} \hat{R}(f^{+x_{k'}})$

- Note, computing $\arg\min_{k'} \hat{R}(f^{+x_{k'}})$ can be done relatively efficiently due to properties of Gaussians (Refer to the paper for details.)

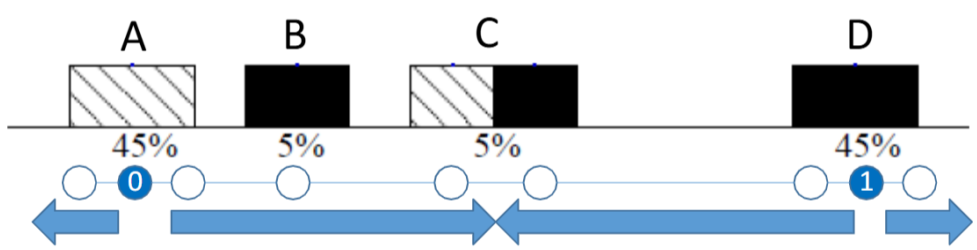
Pseudocode

```
Build a neighborhood graph over data in the pool
Convert the neighborhood graph into a Gaussian Random Field
Iteratively
    Infer labels using the model
    Greedily select the point expected to minimize future risk
    Call oracle to get true label for selected point
    Update the model
```

Limitations of the ZLG algorithm

- The algorithm requires $O(n^3)$ work per iteration because it performs matrix inversion, which could be a problem for large n

- Sampling bias:



- Suppose we have these observations. The inference procedure effectively implements a diffusion process, and the most uncertain points will lie at or near the mid-point. The model is over-confident and will continue to suffer from sample bias until a point from region B is selected.